

Java - Introduction to Programming

Lecture 1

Installation & First Program

1. Install Java

a. Install JDK (<https://www.oracle.com/in/java/technologies/javase-downloads.html>)

b. Install IntelliJ (<https://www.jetbrains.com/idea/download/#section=mac>)

OR

b. Install Visual Studio Code (VS Code) - Prefer THIS

(<https://code.visualstudio.com/download>)

2. Sample Code

Functions

A function is a block of code which takes some input, performs some operations and returns some output.

The functions stored inside classes are called methods.

The function we have used is called main.

Class

A class is a group of objects which have common properties. A class can have some properties and functions (called methods).

The class we have used is Main.

3. Our 1st Program

```
public class Main {  
  
    public static void main(String[] args) {  
        // Our 1st Program  
        System.out.println("Hello World");  
    }  
}
```

Java - Introduction to Programming

Lecture 2

Variables & Data Types

1. Variables

A variable is a container (storage area) used to hold data. Each variable should be given a unique name (identifier).

```
package com.apnacollege;  
  
public class Main {  
  
    public static void main(String[] args) {  
        // Variables  
        String name = "Aman";  
        int age = 30;  
  
        String neighbour = "Akku";  
        String friend = neighbour;  
    }  
}
```

2. Data Types

Data types are declarations for variables. This determines the type and size of data associated with variables which is essential to know since different data types occupy different sizes of memory.

There are 2 types of Data Types :

- Primitive Data types : to store simple values
- Non-Primitive Data types : to store complex values

Primitive Data Types

These are the data types of fixed size.

Data Type	Meaning	Size (in Bytes)	Range
byte	2's complement integer	1	-128 to 127
short	2's complement integer	2	-32K to 32K
int	Integer numbers	4	-2B to 2B
long	2's complement integer (larger values)	8	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
float	Floating-point	4	Upto 7 decimal digits
double	Double Floating-point	8	Upto 16 decimal digits
char	Character	2	a, b, c .. A, B, C .. @, #, \$..
bool	Boolean	1	True, false

Non-Primitive Data Types

These are of variable size & are usually declared with a 'new' keyword.

Eg : String, Arrays

```
String name = new String("Aman");
int[] marks = new int[3];
marks[0] = 97;
marks[1] = 98;
marks[2] = 95;
```

3. Constants

A constant is a variable in Java which has a fixed value i.e. it cannot be assigned a different value once assigned.

Java - Introduction to Programming

Lecture 2

Variables & Data Types

1. Variables

A variable is a container (storage area) used to hold data. Each variable should be given a unique name (identifier).

```
package com.apnacollege;  
  
public class Main {  
  
    public static void main(String[] args) {  
        // Variables  
        String name = "Aman";  
        int age = 30;  
  
        String neighbour = "Akku";  
        String friend = neighbour;  
    }  
}
```

2. Data Types

Data types are declarations for variables. This determines the type and size of data associated with variables which is essential to know since different data types occupy different sizes of memory.

There are 2 types of Data Types :

- Primitive Data types : to store simple values
- Non-Primitive Data types : to store complex values

Primitive Data Types

These are the data types of fixed size.

Data Type	Meaning	Size (in Bytes)	Range
byte	2's complement integer	1	-128 to 127
short	2's complement integer	2	-32K to 32K
int	Integer numbers	4	-2B to 2B
long	2's complement integer (larger values)	8	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
float	Floating-point	4	Upto 7 decimal digits
double	Double Floating-point	8	Upto 16 decimal digits
char	Character	2	a, b, c .. A, B, C .. @, #, \$..
bool	Boolean	1	True, false

Non-Primitive Data Types

These are of variable size & are usually declared with a 'new' keyword.

Eg : String, Arrays

```
String name = new String("Aman");
int[] marks = new int[3];
marks[0] = 97;
marks[1] = 98;
marks[2] = 95;
```

3. Constants

A constant is a variable in Java which has a fixed value i.e. it cannot be assigned a different value once assigned.

```
package com.apnacollege;  
  
public class Main {  
  
    public static void main(String[] args) {  
        // Constants  
        final float PI = 3.14F;  
    }  
}
```

Homework Problems

1. Try to declare meaningful variables of each type. Eg – a variable named age should be a numeric type (int or float) not byte.
2. Make a program that takes the radius of a circle as input, calculates its radius and area and prints it as output to the user.
3. Make a program that prints the table of a number that is input by the user.

(HINT – You will have to write 10 lines for this but as we proceed in the course you will be studying about 'LOOPS' that will simplify your work A LOT!)

KEEP LEARNING & KEEP PRACTICING :)

Java - Introduction to Programming

Lecture 3

1. Conditional Statements 'if-else'

The if block is used to specify the code to be executed if the condition specified in if is true, the else block is executed otherwise.

```
int age = 30;
if (age > 18) {
    System.out.println("This is an adult");
} else {
    System.out.println("This is not an adult");
}
```

2. Conditional Statements 'switch'

Switch case statements are a substitute for long if statements that compare a variable to multiple values. After a match is found, it executes the corresponding code of that value case.

The following example is to print days of the week:

```
int n = 1;
switch (n) {
    case 1 :
        System.out.println("Monday");
        break;
    case 2 :
        System.out.println("Tuesday");
        break;
    case 3 :
        System.out.println("Wednesday");
        break;
    case 4 :
        System.out.println("Thursday");
        break;
    case 5 :
        System.out.println("Friday");
        break;
    case 6 :
        System.out.println("Saturday");
        break;
    default :
        System.out.println("Sunday");
}
```

Homework Problems

1. Make a Calculator. Take 2 numbers (a & b) from the user and an operation as follows :

1 : + (Addition) $a + b$

- 2 : - (Subtraction) $a - b$
- 3 : * (Multiplication) $a * b$
- 4 : / (Division) a / b
- 5 : % (Modulo or remainder) $a \% b$

Calculate the result according to the operation given and display it to the user.

2. Ask the user to enter the number of the month & print the name of the month. For eg – For '1' print 'January', '2' print 'February' & so on.

KEEP LEARNING & KEEP PRACTICING :)

Java - Introduction to Programming

Lecture 4

Loops

A loop is used for executing a block of statements repeatedly until a particular condition is satisfied. A loop consists of an initialization statement, a test condition and an increment statement.

For Loop

The syntax of the for loop is :

```
for (initialization; condition; update) {  
    // body of-loop  
}
```

```
for (int i=1; i<=20; i++) {  
    System.out.println(i);  
}
```

While Loop

The syntax for while loop is :

```
while(condition) {  
    // body of the loop  
}
```

```
int i = 0;  
while (i<=20) {  
    System.out.println(i);  
    i++;  
}
```

Do-While Loop

The syntax for the do-while loop is :

```
do {  
    // body of loop;  
}  
while (condition);
```

```
int i = 0;  
do {  
    System.out.println(i);  
}
```

```
i++;  
} while(i<=20);
```

Homework Problems

1. Print all even numbers till n.
2. Run

```
for(;;) {  
  
    System.out.println(" ");  
  
}
```

loop on your system and analyze what happens. Try to think of the reason for the output produced.

3. Make a menu driven program. The user can enter 2 numbers, either 1 or 0.

If the user enters 1 then keep taking input from the user for a student's marks(out of 100).

If they enter 0 then stop.

If he/ she scores :

Marks >=90 -> print "This is Good"

89 >= Marks >= 60 -> print "This is also Good"

59 >= Marks >= 0 -> print "This is Good as well"

Because marks don't matter but our effort does.

(Hint : use do-while loop but think & understand why)

BONUS

Qs. Print if a number is prime or not (Input n from the user).

[In this problem you will learn how to check if a number is prime or not]

Homework Solution (Lecture 3)

```
import java.util.*;

public class Conditions {

    public static void main(String args[]) {

        Scanner sc = new Scanner(System.in);

        int a = sc.nextInt();
        int b = sc.nextInt();
        int operator = sc.nextInt();

        /**
         * 1 -> +
         * 2 -> -
         * 3 -> *
         * 4 -> /
         * 5 -> %
         */

        switch(operator) {

            case 1 : System.out.println(a+b);
                    break;

            case 2 : System.out.println(a-b);
                    break;

            case 3 : System.out.println(a*b);
                    break;

            case 4 : if(b == 0) {
                        System.out.println("Invalid Division");
                    } else {
                        System.out.println(a/b);
                    }

                    break;

            case 5 : if(b == 0) {
                        System.out.println("Invalid Division");
                    } else {
                        System.out.println(a%b);
                    }

                    break;

            default : System.out.println("Invalid Operator");

        }

    }

}
```

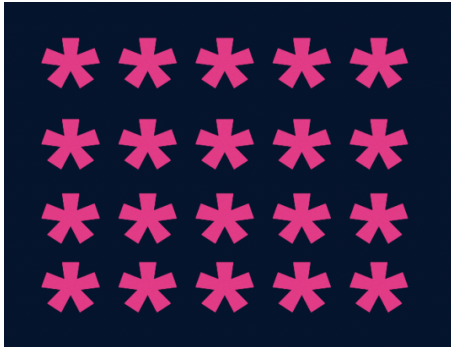
}

Java - Introduction to Programming

Lecture 5

Patterns - Part 1

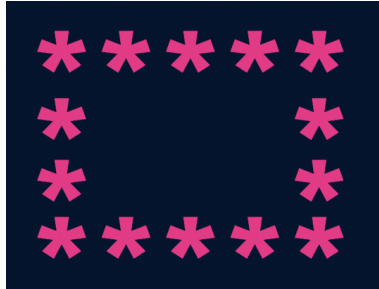
1.



```
import java.util.*;

public class Patterns {
    public static void main(String args[]) {
        int n = 5;
        int m = 4;
        for(int i=0; i<n; i++) {
            for(int j=0; j<m; j++) {
                System.out.print("*");
            }
            System.out.println();
        }
    }
}
```

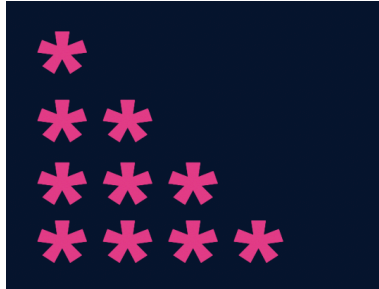
2.



```
import java.util.*;

public class Patterns {
    public static void main(String args[]) {
        int n = 5;
        int m = 4;
        for(int i=0; i<n; i++) {
            for(int j=0; j<m; j++) {
                if(i == 0 || i == n-1 || j == 0 || j == m-1) {
                    System.out.print("*");
                } else {
                    System.out.print(" ");
                }
            }
            System.out.println();
        }
    }
}
```

3.

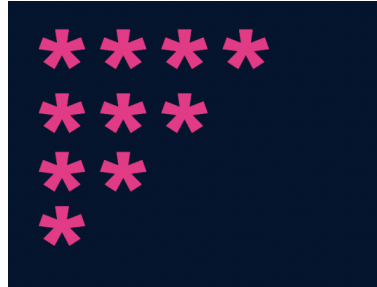


```
import java.util.*;

public class Patterns {
    public static void main(String args[]) {
        int n = 4;

        for(int i=1; i<=n; i++) {
            for(int j=1; j<=i; j++) {
                System.out.print("*");
            }
            System.out.println();
        }
    }
}
```

4.

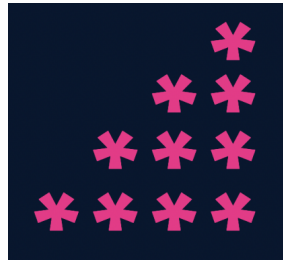


```
import java.util.*;

public class Patterns {
    public static void main(String args[]) {
        int n = 4;

        for(int i=n; i>=1; i--) {
            for(int j=1; j<=i; j++) {
                System.out.print("*");
            }
            System.out.println();
        }
    }
}
```


5.



```
import java.util.*;

public class Patterns {
    public static void main(String args[]) {
        int n = 4;

        for(int i=n; i>=1; i--) {
            for(int j=1; j<i; j++) {
                System.out.print(" ");
            }

            for(int j=0; j<=n-i; j++) {
                System.out.print("*");
            }

            System.out.println();
        }
    }
}
```

6.

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

```
import java.util.*;

public class Patterns {
    public static void main(String args[]) {
        int n = 5;

        for(int i=1; i<=n; i++) {
            for(int j=1; j<=i; j++) {
                System.out.print(j);
            }
            System.out.println();
        }
    }
}
```

7.

```
1 2 3 4 5
1 2 3 4
1 2 3
1 2
1
```

```
import java.util.*;

public class Patterns {
    public static void main(String args[]) {
        int n = 5;

        for(int i=n; i>=1; i--) {
            for(int j=1; j<=i; j++) {
                System.out.print(j);
            }
            System.out.println();
        }
    }
}
```

8.

```
1
2 3
4 5 6
7 8 9 10
11 12 13 14
```

```
import java.util.*;

public class Patterns {
    public static void main(String args[]) {
        int n = 5;
        int number = 1;

        for(int i=1; i<=n; i++) {
            for(int j=1; j<=i; j++) {
                System.out.print(number+" ");
                number++;
            }
            System.out.println();
        }
    }
}
```

9.

```
1
01
101
0101
01010
```

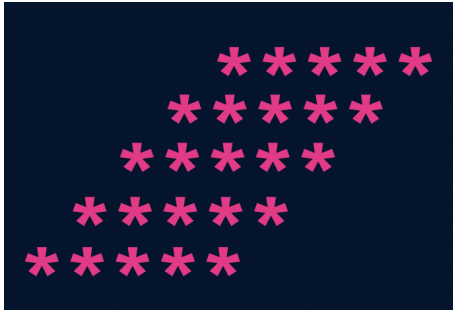
```
import java.util.*;

public class Patterns {
    public static void main(String args[]) {
        int n = 5;

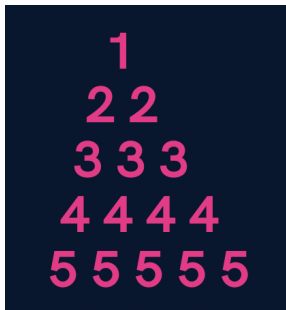
        for(int i=1; i<=n; i++) {
            for(int j=1; j<=i; j++) {
                if((i+j) % 2 == 0) {
                    System.out.print(1+" ");
                } else {
                    System.out.print(0+" ");
                }
            }
            System.out.println();
        }
    }
}
```

Homework Problems (Solutions in next Lecture's Video)

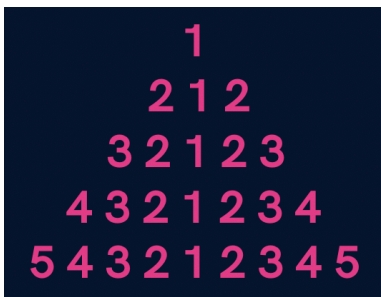
1. Print a solid rhombus.



2. Print a number pyramid.



3. Print a palindromic number pyramid.



Homework Solution (Lecture 4)

1. Print all even numbers till n.

```
1. public class Solutions {  
2.     public static void main(String args[]) {  
3.         int n = 25;  
4.  
5.         for(int i=1; i<=n; i++) {  
6.             if(i % 2 == 0) {  
7.                 System.out.println(i);  
8.             }  
9.         }  
10.     }  
11. }  
12.
```

3. Make a menu driven program. The user can enter 2 numbers, either 1 or 0.
If the user enters 1 then keep taking input from the user for a student's marks(out of 100).

If they enter 0 then stop.

If he/ she scores :

Marks >=90 -> print "This is Good"

89 >= Marks >= 60 -> print "This is also Good"

59 >= Marks >= 0 -> print "This is Good as well"

Because marks don't matter but our effort does.

(Hint : use do-while loop but think & understand why)

```
import java.util.*;

public class Solutions {
    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);
        int input;

        do {
            int marks = sc.nextInt();
            if(marks >= 90 && marks <= 100) {
                System.out.println("This is Good");
            } else if(marks >= 60 && marks <= 89) {
                System.out.println("This is also Good");
            } else if(marks >= 0 && marks <= 59) {
                System.out.println("This is Good as well");
            } else {
                System.out.println("Invalid");
            }

            System.out.println("Want to continue ? (yes(1) or no(0))");
            input = sc.nextInt();

        } while(input == 1);
    }
}
```

Qs. Print if a number n is prime or not (Input n from the user).

[In this problem you will learn how to check if a number is prime or not]

```
import java.util.*;

public class Solutions {
    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();

        boolean isPrime = true;
        for(int i=2; i<=n/2; i++) {
```



```
        if(n % i == 0) {
            isPrime = false;
            break;
        }
    }

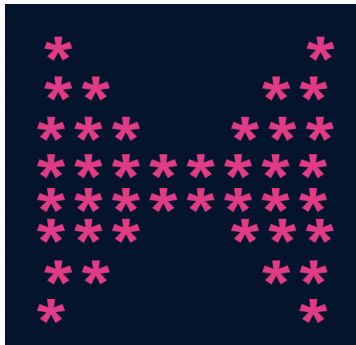
    if(isPrime) {
        if(n == 1) {
            System.out.println("This is neither prime not composite");
        } else {
            System.out.println("This is a prime number");
        }
    } else {
        System.out.println("This is not a prime number");
    }
}
}
```

Java - Introduction to Programming

Lecture 6

Patterns - Part 2

1.



```
import java.util.*;

public class Solutions {
    public static void main(String args[]) {
        int n = 4;

        //upper part
        for(int i=1; i<=n; i++) {
            for(int j=1; j<=i; j++) {
                System.out.print("*");
            }

            int spaces = 2 * (n-i);
            for(int j=1; j<=spaces; j++) {
                System.out.print(" ");
            }

            for(int j=1; j<=i; j++) {
                System.out.print("*");
            }
            System.out.println();
        }
    }
}
```

```

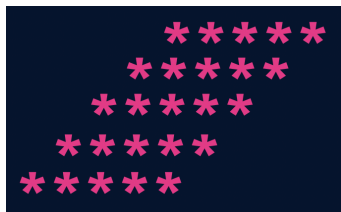
//lower part
for(int i=n; i>=1; i--) {
    for(int j=1; j<=i; j++) {
        System.out.print("*");
    }

    int spaces = 2 * (n-i);
    for(int j=1; j<=spaces; j++) {
        System.out.print(" ");
    }

    for(int j=1; j<=i; j++) {
        System.out.print("*");
    }
    System.out.println();
}
}
}

```

2.



```

import java.util.*;

public class Solutions {
    public static void main(String args[]) {
        int n = 5;
    }
}

```

```
for(int i=1; i<=n; i++) {  
    //spaces  
    for(int j=1; j<=n-i; j++) {  
        System.out.print(" ");  
    }  
  
    //stars  
    for(int j=1; j<=n; j++) {  
        System.out.print("*");  
    }  
    System.out.println();  
}  
}
```

3.

```

  1
 2 2
3 3 3
4 4 4 4
5 5 5 5 5

```

```
import java.util.*;

public class Solutions {
    public static void main(String args[]) {
        int n = 5;

        for(int i=1; i<=n; i++) {
            //spaces
            for(int j=1; j<=n-i; j++) {
                System.out.print(" ");
            }

            //numbers
            for(int j=1; j<=i; j++) {
                System.out.print(i+" ");
            }
            System.out.println();
        }
    }
}
```

4.

```

    1
  2 1 2
3 2 1 2 3
4 3 2 1 2 3 4
5 4 3 2 1 2 3 4 5
```

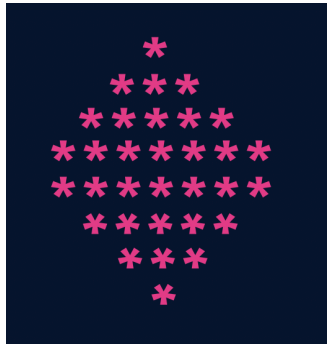
```
import java.util.*;

public class Solutions {
    public static void main(String args[]) {
        int n = 5;
        for(int i=1; i<=n; i++) {
            //spaces
            for(int j=1; j<=n-i; j++) {
                System.out.print(" ");
            }

            //first part
            for(int j=i; j>=1; j--) {
                System.out.print(j);
            }

            //second part
            for(int j=2; j<=i; j++) {
                System.out.print(j);
            }
            System.out.println();
        }
    }
}
```

5.



```
import java.util.*;

public class Solutions {
    public static void main(String args[]) {
        int n = 5;

        //upper part
        for(int i=1; i<=n; i++) {
            //spaces
            for(int j=1; j<=n-i; j++) {
                System.out.print(" ");
            }
            for(int j=1; j<=2*i-1; j++) {
                System.out.print("*");
            }
            System.out.println();
        }

        //lower part
        for(int i=n; i>=1; i--) {
            //spaces
            for(int j=1; j<=n-i; j++) {
                System.out.print(" ");
            }
            for(int j=1; j<=2*i-1; j++) {
                System.out.print("*");
            }
            System.out.println();
        }
    }
}
```

Homework Problems

1. Print a hollow Butterfly.



2. Print a hollow Rhombus.

```
*****
*      *
*      *
*      *
*      *
*****
```

3. Print Pascal's Triangle.

```
1
11
121
1331
14641
```

4. Print half Pyramid.

```
1
```


1 2

1 2 3

1 2 3 4

1 2 3 4 5

5. Print Inverted Half Pyramid.

1111

222

33

4

Java - Introduction to Programming

Lecture 7

Methods/Functions

A function is a block of code that performs a specific task.

Why are functions used?

- a. If some functionality is performed at multiple places in software, then rather than writing the same code, again and again, we create a function and call it everywhere. This helps reduce code redundancy.
- b. Functions make maintenance of code easy as we have to change at one place if we make future changes to the functionality.
- c. Functions make the code more readable and easy to understand.

The **syntax** for function declaration is :

```
return-type function_name (parameter 1, parameter2, ..... parameter n){  
    //function_body  
}  
return-type
```

The **return type** of a function is the data type of the variable that that function returns.

For eg - If we write a function that adds 2 integers and returns their sum then the return type of this function will be 'int' as we will return a sum that is an integer value.

When a function does not return any value, in that case the return type of the function is 'void'.

function_name

It is the unique name of that function.

It is always recommended to declare a function before it is used.

Parameters

A function can take some parameters as inputs. These parameters are specified along with their data types.

For eg- if we are writing a function to add 2 integers, the parameters would be passed like –

```
int add (int num1, int num2)
```

main function

The main function is a special function as the computer starts running the code from the beginning of the main function. Main function serves as the entry point for the program.

Example :

```
package com.apnacollege;

public class Main {
    //A METHOD to calculate sum of 2 numbers - a & b
    public static void sum(int a, int b) {
        int sum = a + b;
        System.out.println(sum);
    }

    public static void main(String[] args) {
        int a = 10;
        int b = 20;
        sum(a, b); // Function Call
    }
}
```

Qs. Write a function to multiply 2 numbers.

```
import java.util.*;

public class Functions {

    //Multiply 2 numbers

    public static int multiply(int a, int b) {

        return a*b;

    }

    public static void main(String args[]) {

        Scanner sc = new Scanner(System.in);
```

```
int a = sc.nextInt();

int b = sc.nextInt();

int result = multiply(a, b);

System.out.println(result);

}

}
```

Qs. Write a function to calculate the factorial of a number.

```
import java.util.*;

public class Functions {

    // public static int calculateSum(int a, int b) {
    //     int sum = a + b;
    //     return sum;
    // }

    // public static int calculateProduct(int a, int b) {
    //     return a * b;
    // }

    public static void printFactorial(int n) {
        //loop
        if(n < 0) {
            System.out.println("Invalid Number");
            return;
        }
        int factorial = 1;

        for(int i=n; i>=1; i--) {
            factorial = factorial * i;
        }

        System.out.println(factorial);
        return;
    }
}
```

```
public static void main(String args[]) {  
    Scanner sc = new Scanner(System.in);  
    int n = sc.nextInt();  
  
    printFactorial(n);  
}  
}
```

Qs. Write a function to calculate the product of 2 numbers.

```
import java.util.*;  
  
public class Functions {  
  
    // public static int calculateSum(int a, int b) {  
  
        // int sum = a + b;  
  
        // return sum;  
  
    // }  
  
    public static int calculateProduct(int a, int b) {  
  
        return a * b;  
  
    }  
  
    public static void main(String args[]) {  
  
        Scanner sc = new Scanner(System.in);  
  
        int a = sc.nextInt();  
  
        int b = sc.nextInt();  
  
        System.out.println(calculateProduct(a, b));  
  
    }  
}
```

Homework Problems

1. Make a function to check if a number is prime or not.
2. Make a function to check if a given number n is even or not.
3. Make a function to print the table of a given number n .
4. Read about Recursion.

Java - Introduction to Programming

Lecture 10

Arrays In Java

Arrays in Java are like a list of elements of the same type i.e. a list of integers, a list of booleans etc.

- a. Creating an Array (method 1) - with **new** keyword

```
int[] marks = new int[3];  
marks[0] = 97;  
marks[1] = 98;  
marks[2] = 95;
```

- b. Creating an Array (method 2)

```
int[] marks = {98, 97, 95};
```

- c. Taking an array as an input and printing its elements.

```
import java.util.*;  
  
public class Arrays {  
    public static void main(String args[]) {  
        Scanner sc = new Scanner(System.in);  
        int size = sc.nextInt();  
        int numbers[] = new int[size];  
  
        for(int i=0; i<size; i++) {  
            numbers[i] = sc.nextInt();  
        }  
  
        //print the numbers in array  
        for(int i=0; i<arr.length; i++) {  
            System.out.print(numbers[i]+" ");  
        }  
    }  
}
```

Homework Problems

1. Take an array of names as input from the user and print them on the screen.

```
import java.util.*;

public class Arrays {

    public static void main(String args[]) {

        Scanner sc = new Scanner(System.in);

        int size = sc.nextInt();

        String names[] = new String[size];

        //input

        for(int i=0; i<size; i++) {

            names[i] = sc.next();

        }

        //output

        for(int i=0; i<names.length; i++) {

            System.out.println("name " + (i+1) + " is : " + names[i]);

        }

    }

}
```

2. Find the maximum & minimum number in an array of integers.

[HINT : Read about [Integer.MIN_VALUE](#) & [Integer.MAX_VALUE](#) in Java]

```
import java.util.*;

public class Arrays {

    public static void main(String args[]) {

        Scanner sc = new Scanner(System.in);

        int size = sc.nextInt();

        int numbers[] = new int[size];

        //input

        for(int i=0; i<size; i++) {

            numbers[i] = sc.nextInt();

        }

        int max = Integer.MIN_VALUE;

        int min = Integer.MAX_VALUE;

        for(int i=0; i<numbers.length; i++) {

            if(numbers[i] < min) {

                min = numbers[i];

            }

            if(numbers[i] > max) {

                max = numbers[i];

            }

        }

    }

}
```

```

        System.out.println("Largest number is : " + max);

        System.out.println("Smallest number is : " + min);

    }
}

```

3. Take an array of numbers as input and check if it is an array sorted in ascending order.

Eg: { 1, 2, 4, 7 } is sorted in ascending order.

{3, 4, 6, 2} is not sorted in ascending order.

```

import java.util.*;

public class Arrays {

    public static void main(String args[]) {

        Scanner sc = new Scanner(System.in);

        int size = sc.nextInt();

        int numbers[] = new int[size];

        //input

        for(int i=0; i<size; i++) {

            numbers[i] = sc.nextInt();

        }

        boolean isAscending = true;
    }
}

```

```
        for(int i=0; i<numbers.length-1; i++) { // NOTICE numbers.length - 1 as
termination condition

            if(numbers[i] > numbers[i+1]) { // This is the condition for
descending order

                isAscending = false;

            }

        }

        if(isAscending) {

            System.out.println("The array is sorted in ascending order");

        } else {

            System.out.println("The array is not sorted in ascending order");

        }

    }

}
```

Java - Introduction to Programming

Lecture 11

2D Arrays In Java

It is similar to 2D matrices that we studied in 11th and 12th class.

- a. Creating a 2D Array - with **new** keyword

```
int[][] marks = new int[3][3];
```

- b. Taking a matrix as an input and printing its elements.

```
import java.util.*;

public class TwoDArrays {
    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);
        int rows = sc.nextInt();
        int cols = sc.nextInt();

        int[][] numbers = new int[rows][cols];

        //input
        //rows
        for(int i=0; i<rows; i++) {
            //columns
            for(int j=0; j<cols; j++) {
                numbers[i][j] = sc.nextInt();
            }
        }

        for(int i=0; i<rows; i++) {
            for(int j=0; j<cols; j++) {
                System.out.print(numbers[i][j]+" ");
            }
            System.out.println();
        }
    }
}
```

```
}
```

c. Searching for an element x in a matrix.

```
import java.util.*;

public class TwoDArrays {
    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);
        int rows = sc.nextInt();
        int cols = sc.nextInt();

        int[][] numbers = new int[rows][cols];

        //input
        //rows
        for(int i=0; i<rows; i++) {
            //columns
            for(int j=0; j<cols; j++) {
                numbers[i][j] = sc.nextInt();
            }
        }

        int x = sc.nextInt();

        for(int i=0; i<rows; i++) {
            for(int j=0; j<cols; j++) {
                //compare with x
                if(numbers[i][j] == x) {
                    System.out.println("x found at location (" + i + ", " + j +
                    ")");
                }
            }
        }
    }
}
```

Homework Problems

1. Print the spiral order matrix as output for a given matrix of numbers.
[Difficult for Beginners]

For example: for the given matrix,

1	5	7	9	10	11
6	10	12	13	20	21
9	25	29	30	32	41
15	55	59	63	68	70
40	70	79	81	95	105

Spiral order is given by:

1 5 7 9 10 11 21 41 70 105 95 81 79 70 40 15 9 6 10 12 13 20 32 68 63 59 55
25 29 30 29.

APPROACH :

Algorithm: (We are given a 2D matrix of $n \times m$).

1. We will need 4 variables:

a. *row_start* – initialized with 0.

b. *row_end* – initialized with $n-1$.

c. *column_start* – initialized with 0.

d. *column_end* – initialized with $m-1$.

2. First of all, we will traverse in the row *row_start* from *column_start*

to `column_end` and we will increase the `row_start` with 1 as we have traversed the starting row.

3. Then we will traverse in the column `column_end` from `row_start` to `row_end` and decrease the `column_end` by 1.

4. Then we will traverse in the row `row_end` from `column_end` to `column_start` and decrease the `row_end` by 1.

5. Then we will traverse in the column `column_start` from `row_end` to `row_start` and increase the `column_start` by 1.

6. We will do the above steps from 2 to 5 until `row_start <= row_end` and `column_start <= column_end`.

```
import java.util.*;

public class Arrays {

    public static void main(String args[]) {

        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        int m = sc.nextInt();

        int matrix[][] = new int[n][m];

        for(int i=0; i<n; i++) {

            for(int j=0; j<m; j++) {

                matrix[i][j] = sc.nextInt();

            }

        }

        System.out.println("The Spiral Order Matrix is : ");

        int rowStart = 0;
```

```
int rowEnd = n-1;

int colStart = 0;

int colEnd = m-1;

//To print spiral order matrix

while(rowStart <= rowEnd && colStart <= colEnd) {

    //1

    for(int col=colStart; col<=colEnd; col++) {

        System.out.print(matrix[rowStart][col] + " ");

    }

    rowStart++;

    //2

    for(int row=rowStart; row<=rowEnd; row++) {

        System.out.print(matrix[row][colEnd] + " ");

    }

    colEnd--;

    //3

    for(int col=colEnd; col>=colStart; col--) {

        System.out.print(matrix[rowEnd][col] + " ");

    }

    rowEnd--;

    //4

    for(int row=rowEnd; row>=rowStart; row--) {
```



```

        System.out.print(matrix[row][colStart] + " ");

    }

    colStart++;

    System.out.println();

}

}

}

```

2. For a given matrix of N x M, print its transpose.

```

import java.util.*;

public class Arrays {

    public static void main(String args[]) {

        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        int m = sc.nextInt();

        int matrix[][] = new int[n][m];

        for(int i=0; i<n; i++) {

            for(int j=0; j<m; j++) {

                matrix[i][j] = sc.nextInt();

            }

        }

    }

}

```

```
System.out.println("The transpose is : ");

//To print transpose

for(int j=0; j<m ;j++) {

    for(int i=0; i<n; i++) {

        System.out.print(matrix[i][j]+" ");

    }

    System.out.println();

}

}
```

Java - Introduction to Programming

Lecture 12

Strings

Declaration

```
String name = "Tony";
```

Taking Input

```
Scanner sc = new Scanner(System.in);  
  
String name = sc.next();
```

Concatenation (Joining 2 strings)

```
String firstName = "Tony";  
String secondName = "Stark";  
  
String fullName = firstName + " " + secondName;  
System.out.println(fullName);
```

Print length of a String

```
String firstName = "Tony";  
String secondName = "Stark";  
  
String fullName = firstName + " " + secondName;  
System.out.println(fullName.length());
```

Access characters of a string

```
String firstName = "Tony";  
String secondName = "Stark";  
  
String fullName = firstName + " " + secondName;  
  
for(int i=0; i<fullName.length(); i++) {  
    System.out.println(fullName.charAt(i));  
}
```

Compare 2 strings

```
import java.util.*;

public class Strings {
    public static void main(String args[]) {
        String name1 = "Tony";
        String name2 = "Tony";

        if(name1.equals(name2)) {
            System.out.println("They are the same string");
        } else {
            System.out.println("They are different strings");
        }

        //DO NOT USE == to check for string equality
        //Gives correct answer here
        if(name1 == name2) {
            System.out.println("They are the same string");
        } else {
            System.out.println("They are different strings");
        }

        //Gives incorrect answer here
        if(new String("Tony") == new String("Tony")) {
            System.out.println("They are the same string");
        } else {
            System.out.println("They are different strings");
        }
    }
}
```

Substring

The substring of a string is a subpart of it.

```
public class Strings {
    public static void main(String args[]) {
        String name = "TonyStark";

        System.out.println(name.substring(0, 4));
    }
}
```

```
}  
}
```

parseInt Method of Integer class

```
public class Strings {  
    public static void main(String args[]) {  
        String str = "123";  
        int number = Integer.parseInt(str);  
        System.out.println(number);  
  
    }  
}
```

ToString Method of String class

```
public class Strings {  
    public static void main(String args[]) {  
        int number = 123;  
        String str = Integer.toString(number);  
        System.out.println(str.length());  
  
    }  
}
```

ALWAYS REMEMBER : Java Strings are Immutable.

Homework Problems

1. Take an array of Strings input from the user & find the cumulative (combined) length of all those strings.

```
import java.util.*;

public class Strings {

    public static void main(String args[]) {

        Scanner sc = new Scanner (System.in);

        int size = sc.nextInt();

        String array[] = new String[size];

        int totLength = 0;

        for(int i=0; i<size; i++) {

            array[i] = sc.next();

            totLength += array[i].length();

        }

        System.out.println(totLength);

    }

}
```

2. Input a string from the user. Create a new string called 'result' in which you will replace the letter 'e' in the original string with letter 'i'.

Example :

original = "eabcdef" ; result = "iabcdif"

Original = "xyz" ; result = "xyz"

```

import java.util.*;

public class Strings {

    public static void main(String args[]) {

        Scanner sc = new Scanner (System.in);

        String str = sc.next();

        String result = "";

        for(int i=0; i<str.length(); i++) {

            if(str.charAt(i) == 'e') {

                result += 'i';

            } else {

                result += str.charAt(i);

            }

        }

        System.out.println(result);

    }

}

```

3. Input an email from the user. You have to create a username from the email by deleting the part that comes after '@'. Display that username to the user.

Example :

email = "apnaCollegeJava@gmail.com"; username = "apnaCollegeJava"

email = "helloWorld123@gmail.com"; username = "helloWorld123"

```
import java.util.*;

public class Strings {

    public static void main(String args[]) {

        Scanner sc = new Scanner (System.in);

        String email = sc.next();

        String userName = "";

        for(int i=0; i<email.length(); i++) {

            if(email.charAt(i) == '@') {

                break;

            } else {

                userName += email.charAt(i);

            }

        }

        System.out.println(userName);

    }

}
```


Java - Introduction to Programming

Lecture 13

String Builder

Declaration

```
StringBuilder sb = new StringBuilder(" ");  
System.out.println(sb);
```

Get A Character from Index

```
StringBuilder sb = new StringBuilder("Tony");  
//Set Char  
System.out.println(sb.charAt(0));
```

Set a Character at Index

```
StringBuilder sb = new StringBuilder("Tony");  
//Get Char  
sb.setCharAt(0, 'P');  
System.out.println(sb);
```

Insert a Character at Some Index

```
import java.util.*;  
  
public class Strings {  
    public static void main(String args[]) {  
        StringBuilder sb = new StringBuilder("tony");  
        //Insert char  
        sb.insert(0, 'S');  
        System.out.println(sb);  
    }  
}
```

Delete char at some Index

```
import java.util.*;

public class Strings {
    public static void main(String args[]) {
        StringBuilder sb = new StringBuilder("tony");
        //Insert char
        sb.insert(0, 'S');
        System.out.println(sb);

        //delete char
        sb.delete(0, 1);
        System.out.println(sb);
    }
}
```

Append a char

Append means to add something at the end.

```
import java.util.*;

public class Strings {
    public static void main(String args[]) {
        StringBuilder sb = new StringBuilder("Tony");
        sb.append(" Stark");
        System.out.println(sb);
    }
}
```

Print Length of String

```
import java.util.*;

public class Strings {
    public static void main(String args[]) {
        StringBuilder sb = new StringBuilder("Tony");
        sb.append(" Stark");
        System.out.println(sb);

        System.out.println(sb.length());
    }
}
```

Reverse a String (using StringBuilder class)

```
import java.util.*;

public class Strings {
    public static void main(String args[]) {
        StringBuilder sb = new StringBuilder("HelloWorld");

        for(int i=0; i<sb.length()/2; i++) {
            int front = i;
            int back = sb.length() - i - 1;

            char frontChar = sb.charAt(front);
            char backChar = sb.charAt(back);

            sb.setCharAt(front, backChar);
            sb.setCharAt(back, frontChar);
        }

        System.out.println(sb);
    }
}
```

Homework Problems

Try Solving all the String questions with StringBuilder.

Java - Introduction to Programming

Lecture 14

Bit Manipulation

Get Bit

```
import java.util.*;

public class Bits {
    public static void main(String args[]) {
        int n = 5; //0101
        int pos = 3;
        int bitMask = 1<<pos;

        if((bitMask & n) == 0) {
            System.out.println("bit was zero");
        } else {
            System.out.println("bit was one");
        }
    }
}
```

Set Bit

```
import java.util.*;

public class Bits {
    public static void main(String args[]) {
        int n = 5; //0101
        int pos = 1;
        int bitMask = 1<<pos;

        int newNumber = bitMask | n;
        System.out.println(newNumber);
    }
}
```

Clear Bit

```
import java.util.*;

public class Bits {

    public static void main(String args[]) {

        int n = 5; //0101

        int pos = 2;

        int bitMask = 1<<pos;

        int newBitMask = ~(bitMask);

        int newNumber = newBitMask & n;

        System.out.println(newNumber);

    }

}
```

Update Bit

```
import java.util.*;

public class Bits {

    public static void main(String args[]) {

        Scanner sc = new Scanner(System.in);

        int oper = sc.nextInt();

        // oper=1 -> set; oper=0 -> clear

        int n = 5;

        int pos = 1;

        int bitMask = 1<<pos;

        if(oper == 1) {

            //set

            int newNumber = bitMask | n;

            System.out.println(newNumber);

        } else {

            //clear

            int newBitMask = ~(bitMask);

            int newNumber = newBitMask & n;

            System.out.println(newNumber);

        }

    }

}
```

Homework Problems

1. Write a program to find if a number is a power of 2 or not.
2. Write a program to toggle a bit a position = "pos" in a number "n".
3. Write a program to count the number of 1's in a binary representation of the number.
4. Write 2 functions => decimalToBinary() & binaryToDecimal() to convert a number from one number system to another. [BONUS]